DATA STRUCTURES LAB PROJECT

P-12.61: Bucket-Sort for Integer Keys

Sorting n entries with keys in the range [0, N−1] in O(n + N) time

PROJECT DATE : 27-10-2026 **SUBMITTED TO : SIR JAMAL**

Project Members

Sr. No	Name	Roll Number
1	MUHAMMAD ABDUL RAHMAN KHAN	25F-DS-106
2	MUHAMMAD HUZAIFA SHAMSI	25F-DS-135
3	BASIT MIRZA	25F-DS-140
4	MUHAMMAD HASHIR	25F-DS-152

Table Of Contents

Sr. No	Heading	Page No
1	Introduction	3
2	Abstract (Objectives and Key Features)	3
3	Methodology (System Design, Data Structures, Algorithms, Development Approach)	4
4	Source Code (with explanation of modules and functions)	6
5	Results (Screenshots, Sample Input/Output, Tests, Graphs)	14
6	Discussion (Analysis, Challenges, Limitations, Lessons Learned)	16
7	Future Work	17
8	Conclusion	17
9	References	18
	Appendix: Source Code (copy and run)	19

Project :

P-12.61: Design and implement a version of the bucket-sort algorithm for sorting a list of n entries with integer keys taken from the range [0, N−1], for N ≥ 2. The algorithm should run in O(n + N) time.

This project is taken from Chapter 12 (Sorting and Selection) of the book "Data Structures and Algorithms in Python" by Goodrich, Tamassia and Goldwasser. Bucket-sort is explained in Section 12.4.2 of the book.

1. Introduction

Sorting means arranging data in an order, for example from smallest to largest. It is one of the most common tasks in programming. We use it for ranking students, showing products by price, or arranging contacts by name. There are many sorting algorithms and each one has its own speed and limits.

Most well-known algorithms like merge sort and quick sort work by comparing two values at a time and deciding which one is bigger. Such algorithms cannot do better than O(n log n) time. Bucket-sort is different. When all keys are small whole numbers, it does not compare keys at all. It uses the key itself to decide where an entry should go. Because of this, it can sort in O(n + N) time, which is faster than any comparison based sort when the range of keys is not too large.

2. Abstract

In this project we designed and implemented bucket-sort in Python for a list of n entries whose keys are integers in the range [0, N−1]. Each entry is stored as a key-value pair. The algorithm creates N empty buckets, places every entry in the bucket that matches its key, and then reads the buckets from 0 to N−1 to get the sorted list. We checked the correctness of our code with four kinds of automated tests. To see how fast it is in practice, we also implemented insertion sort, merge sort and quick sort and measured the running time of all of them. We ran two experiments, one where we increased n and one where we increased N, and showed the results in tables and graphs. The results match the theory: bucket-sort grows linearly with n and is much faster than the other sorts, but it becomes slow when N is much larger than n.

Objectives:

- To implement bucket-sort for integer keys in the range [0, N−1] that runs in O(n + N) time.
- To keep the sort stable, so entries with equal keys stay in their original order.
- To test the code on normal cases, edge cases and wrong input.
- To compare bucket-sort with other sorting algorithms by measuring their running times.
- To find out in which situations bucket-sort is a good choice and when it is not.

Key Features:

- Stable bucket-sort that works on Entry objects (key, value) and also on plain integer lists.
- Input checking: a key outside [0, N−1] or N < 2 gives a clear ValueError.

- Three comparison sorts (insertion, merge and quick sort) for comparison.
- Automated tests that run every time the program starts.
- Two timing experiments with results printed as tables.
- Two graphs drawn with matplotlib, saved as PNG files and shown on screen.

3. Methodology

3.1 System Design:

The whole project is a single Python file, bucket_sort.py. It is divided into small functions so that each part has one job. When the program runs, it first runs the tests, then the two experiments, and at the end it draws the graphs.

Part	Function / Class	Job
Data structure	Entry	Stores one item as a (key, value) pair
Main algorithm	bucket_sort()	Sorts the list using buckets in O(n + N)
Comparison sorts	insertion_sort(), merge_sort(), quick_sort()	Used only to compare running times
Testing	run_tests()	Checks that bucket_sort() gives correct answers
Timing	time_it()	Measures the best of 3 running times of a sort
Experiments	benchmark()	Runs Experiment A and B and prints the tables
Graphs	make_charts()	Draws, saves and shows the two graphs

Flow of the program: run_tests() → benchmark() (uses time_it() for every sort) → make_charts()

3.2 Data Structures Used:

Entry (key, value): Each item to be sorted is an Entry object that holds an integer key and a value. For example, the key can be the marks of a student and the value can be the name. We used __slots__ so that each object uses less memory.

Python list: The input data is a normal Python list. The sorted result is written back into the same list.

Array of buckets: The buckets are stored as a list of N lists. B[k] is the bucket for key k. Each bucket works like a queue: new entries are added at the end with append() and later read from the front. This is what makes our sort stable.

3.3 Algorithms Implemented:

Bucket-sort (main algorithm): Bucket-sort makes one empty bucket for every possible key. It goes through the input once and drops each entry into the bucket with the same number as its key (scatter). Then it goes through the buckets from 0 to N−1 and copies the entries back into the list (gather). Since the buckets are visited in order of key, the list comes out sorted without comparing any two keys. Creating the buckets takes O(N), scatter takes O(n) and gather takes O(n + N), so the total is O(n + N).

Small example with N = 4: input keys 3, 0, 3, 1, 2, 0, 3 → buckets B[0] = 0, 0 · B[1] = 1 · B[2] = 2 · B[3] = 3, 3, 3 → output 0, 0, 1, 2, 3, 3, 3.

Insertion sort: Works the way we arrange playing cards. It picks the next item and moves it left until it reaches its correct place. Worst case $O(n^2)$.

Merge sort: Splits the list into two halves, sorts each half by recursion, and merges the two sorted halves. Always O(n log n).

Quick sort: Picks the middle item as pivot and splits the list into smaller, equal and bigger groups, then sorts the smaller and bigger groups in the same way. O(n log n) on average, $O(n^2)$ in the worst case.

Algorithm	Best	Average	Worst	Compares keys?
Bucket-sort	O(n + N)	O(n + N)	O(n + N)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	Yes

3.4 Development Approach:

We worked step by step:

- First we did a dry run of bucket-sort on paper with a few keys and buckets to understand how it works.
- Then we wrote the Entry class and the bucket_sort() function and tried it on small lists.
- After that we wrote the tests. Whenever we changed the code later, we ran the tests again to make sure nothing was broken.
- Next we added insertion, merge and quick sort, and wrote time_it() and benchmark() to measure running times on random data.
- Finally we added the graphs and studied the results.

Tools used: Python 3, the built-in random and time modules, matplotlib for graphs, and VS Code as the editor.

4. Source Code

The code is organised into sections with comments. Below, the important modules and functions are shown with a short explanation. The complete code is given at the end of this section, and a text version that can be copied and run is in the Appendix.

Entry class:

```
 8 # ----------------------------------------------------------------
 9 # Data structure: one entry = (key, value)
10 # ----------------------------------------------------------------
11 class Entry:
12     __slots__ = ("key", "value")   # only these two fields, saves memory
13
14     def __init__(self, key, value=None):
15         self.key = key
16         self.value = value
17
18     # how an entry looks when printed, e.g. (3, 'c1')
19     def __repr__(self):
20         return f"({self.key}, {self.value!r})"
```

An Entry holds one key and one value. __init__ saves them and __repr__ decides how an entry is printed, for example (3, 'c1'). __slots__ limits the object to these two fields so it uses less memory.

bucket_sort():

```
23 # ----------------------------------------------------------------
24 # Main algorithm: bucket-sort
25 # entries : list to sort (sorted in place)
26 # N       : keys are in the range 0 .. N-1
27 # key     : how to get the key from an item (default: e.key)
28 # ----------------------------------------------------------------
29 def bucket_sort(entries, N, key=lambda e: e.key):
30     if N < 2:
31         raise ValueError("N must be at least 2")
32
33     # create N empty buckets, B[k] holds all entries with key k   -> O(N)
34     B = [[] for _ in range(N)]
35
36     # scatter: put every entry at the end of its bucket           -> O(n)
37     for e in entries:
38         k = key(e)
39         if not (0 <= k < N):
40             raise ValueError(f"key {k} is outside the range [0, {N - 1}]")
41         B[k].append(e)
42
43     # gather: read buckets 0 .. N-1 in order back into the list   -> O(n + N)
44     i = 0
45     for bucket in B:
46         for e in bucket:
47             entries[i] = e
48             i += 1
49
50     return entries
```

The function takes the list, the number N, and a key function. By default the key function reads e.key; for a list of plain numbers we pass key=lambda x: x. First it checks that N is at least 2. Then it creates N empty buckets with a list comprehension. We did not write [[]] * N because that would create N references to the same single list.

In the scatter step (lines 36–41) each entry's key is found, checked to be inside the range, and the entry is appended to bucket B[k]. In the gather step (lines 43–48) the buckets are visited in order and their entries are written back into the original list using the index i. The total time is O(n + N) and the extra memory is also O(n + N).

insertion_sort():

```
57 # insertion sort: shift bigger items right, put cur in the gap   O(n^2)
58 def insertion_sort(a):
59     for j in range(1, len(a)):
60         cur = a[j]
61         i = j
62         while i > 0 and a[i - 1] > cur:
63             a[i] = a[i - 1]
64             i -= 1
65         a[i] = cur
66     return a
```

The outer loop picks each item from index 1 onward and saves it in cur. The while loop shifts bigger items one place to the right as long as the item on the left is greater than cur. Then cur is placed in the empty spot. The condition i > 0 stops the loop at the start of the list.

merge_sort():

```
69 # merge sort: split in halves, sort each half, merge them        O(n log n)
70 def merge_sort(a):
71     if len(a) <= 1:
72         return a[:]
73     mid = len(a) // 2
74     left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
75     # merge two sorted lists, i and j point to the front items
76     out, i, j = [], 0, 0
77     while i < len(left) and j < len(right):
78         if left[i] <= right[j]:
79             out.append(left[i]); i += 1
80         else:
81             out.append(right[j]); j += 1
82     out.extend(left[i:]); out.extend(right[j:])
83     return out
```

A list of 0 or 1 items is already sorted, so a copy is returned. Otherwise the list is split at the middle and both halves are sorted by recursion. While merging, i and j point to the front items of the left and right lists, and the smaller one is added to out. When one side finishes, the rest of the other side is added. Using <= keeps equal items in order.

quick_sort():

```
86 # quick sort: smaller / equal / bigger than the pivot           O(n log n) average
87 def quick_sort(a):
88     if len(a) <= 1:
89         return a[:]
90     pivot = a[len(a) // 2]
91     smaller = [x for x in a if x < pivot]
92     equal = [x for x in a if x == pivot]
93     bigger = [x for x in a if x > pivot]
94     return quick_sort(smaller) + equal + quick_sort(bigger)
```

The middle item is the pivot. Three list comprehensions build the smaller, equal and bigger lists, and the function returns quick_sort(smaller) + equal + quick_sort(bigger). The equal items are never sorted again, which helps when there are many repeated keys.

run_tests():

```
# ---------------------------------------------------------------
# Correctness tests (assert stops the program if an answer is wrong)
# ---------------------------------------------------------------
def run_tests():
    print("=" * 64)
    print("CORRECTNESS TESTS")
    print("=" * 64)

    # Test 1: small example, also checks stability (c1, c2, c3 keep order)
    data = [Entry(3, "c1"), Entry(0, "a"), Entry(3, "c2"), Entry(1, "b"),
            Entry(2, "x"), Entry(0, "a2"), Entry(3, "c3")]
    print("Before :", data)
    bucket_sort(data, 4)
    print("After  :", data)
    assert [e.key for e in data] == [0, 0, 1, 2, 3, 3, 3]
    assert [e.value for e in data if e.key == 3] == ["c1", "c2", "c3"]
    print("Test 1 passed - sorted and STABLE (c1, c2, c3 kept their order)\n")

    # Test 2: 200 random lists compared with Python's sorted()
    for trial in range(200):
        N = random.randint(2, 50)
        n = random.randint(0, 100)
        nums = [random.randrange(N) for _ in range(n)]
        assert bucket_sort(nums[:], N, key=lambda x: x) == sorted(nums)
    print("Test 2 passed - 200 random lists match Python's sorted()")

    # Test 3: edge cases
    assert bucket_sort([], 5, key=lambda x: x) == []
    assert bucket_sort([4], 5, key=lambda x: x) == [4]
    assert bucket_sort([1, 1, 1], 2, key=lambda x: x) == [1, 1, 1]
    assert bucket_sort([4, 3, 2, 1, 0], 5, key=lambda x: x) == [0, 1, 2, 3, 4]
    assert bucket_sort([0, 1, 2, 3], 4, key=lambda x: x) == [0, 1, 2, 3]
    print("Test 3 passed - empty, single, duplicates, reversed, sorted")

    # Test 4: wrong input must raise ValueError
    for bad_call in (lambda: bucket_sort([5], 5, key=lambda x: x),
                     lambda: bucket_sort([-1], 5, key=lambda x: x),
                     lambda: bucket_sort([0], 1, key=lambda x: x)):
        try:
            bad_call()
            raise AssertionError("should have raised ValueError")
        except ValueError:
            pass
    print("Test 4 passed - out-of-range keys and N < 2 raise ValueError\n")
```

Every check uses assert, so if any answer is wrong the program stops. Test 1 sorts a small example and checks stability. Test 2 compares our result with Python's sorted() on 200 random lists; nums[:] passes a copy so that both functions get the same input. Test 3 checks edge cases. In Test 4 each wrong call is wrapped in a lambda so it runs inside try; if no ValueError is raised, the AssertionError line makes the test fail.

time_it():

```
# run func 3 times on a fresh copy of data and return the best time (seconds)
def time_it(func, data, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        copy = data[:]
        start = time.perf_counter()
        func(copy)
        best = min(best, time.perf_counter() - start)
    return best
```

This function runs a sort three times, each time on a fresh copy of the data, and returns the smallest time measured with time.perf_counter(). A fresh copy is needed because after one run the list would already be sorted. The best of three is used because other programs on the computer can only make a run slower.

benchmark():

```
def benchmark():
    random.seed(42)   # same random data on every run
    results = {}

    # Experiment A: N fixed at 1000, number of entries n grows
    print("=" * 64)
    print("EXPERIMENT A: vary n (N = 1000 fixed) - times in milliseconds")
    print("=" * 64)
    N = 1000
    sizes = [1000, 2000, 5000, 10000, 20000, 50000, 100000]
    print(f"{'n':>8} {'Bucket':>10} {'Merge':>10} {'Quick':>10} {'Insertion':>10} {'sorted()':>10}")
    rows_a = []
    for n in sizes:
        data = [random.randrange(N) for _ in range(n)]   # n random keys in 0..N-1
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        tq = time_it(quick_sort, data)
        # insertion sort is O(n^2), so it is only run for small n
        ti = time_it(insertion_sort, data, repeats=1) if n <= 5000 else None
        ts = time_it(sorted, data)
        rows_a.append((n, tb, tm, tq, ti, ts))
        ins = f"{ti*1000:10.2f}" if ti is not None else f"{'(skip)':>10}"
        print(f"{n:>8} {tb*1000:10.2f} {tm*1000:10.2f} {tq*1000:10.2f} {ins} {ts*1000:10.2f}")
    results["A"] = rows_a

    # Experiment B: n fixed at 10000, key range N grows
    print()
    print("=" * 64)
    print("EXPERIMENT B: vary N (n = 10000 fixed) - bucket sort, ms")
    print("=" * 64)
    n = 10000
    print(f"{'N':>10} {'Bucket (ms)':>12} {'Merge (ms)':>12}")
    rows_b = []
    for N in [10, 100, 1000, 10000, 100000, 1000000]:
        data = [random.randrange(N) for _ in range(n)]
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        rows_b.append((N, tb, tm))
        print(f"{N:>10} {tb*1000:12.2f} {tm*1000:12.2f}")
    results["B"] = rows_b
    print()
    return results
```

random.seed(42) makes the random data the same on every run. In Experiment A, N is fixed at 1000 and n grows from 1,000 to 100,000; all five sorts get the same random list in each round, and insertion sort is skipped above n = 5,000 because it is too slow. Bucket-sort needs three arguments, so it is wrapped in a lambda for time_it(). In Experiment B, n is fixed at 10,000 and N grows from 10 to 1,000,000, and bucket-sort is compared with merge sort, which does not depend on N. The f-strings like {x:10.2f} print every value in a fixed width with 2 decimals so the table columns line up.

make_charts() and main program:

```
# draw the two graphs with matplotlib, save them as PNG and show them
def make_charts(results):
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed - charts skipped (pip install matplotlib)")
        return

    # Graph 1: Experiment A (time vs n)
    rows = results["A"]
    n = [r[0] for r in rows]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.plot(n, [r[1] * 1000 for r in rows], "o-", label="Bucket sort  O(n+N)", lw=2.5)
    ax.plot(n, [r[2] * 1000 for r in rows], "s-", label="Merge sort  O(n log n)")
    ax.plot(n, [r[3] * 1000 for r in rows], "^-", label="Quick sort  O(n log n)")
    # insertion sort only has times for small n, skip the None values
    ins = [(r[0], r[4] * 1000) for r in rows if r[4] is not None]
    ax.plot([x for x, _ in ins], [y for _, y in ins], "x--", label="Insertion sort  O(n²)")
    ax.set_xlabel("Number of entries n")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment A: running time vs n  (N = 1000)")
    # limit the y-axis so insertion sort does not squeeze the other lines
    ax.set_ylim(0, max(r[2] for r in rows) * 1000 * 1.15)
    ax.grid(alpha=0.3); ax.legend()
    fig.tight_layout(); fig.savefig("chart_experiment_A.png", dpi=150)

    # Graph 2: Experiment B (time vs N), x-axis on log scale
    rows = results["B"]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.semilogx([r[0] for r in rows], [r[1] * 1000 for r in rows], "o-", lw=2.5, label="Bucket sort")
    ax.semilogx([r[0] for r in rows], [r[2] * 1000 for r in rows], "s-", label="Merge sort")
    ax.set_xlabel("Key range N (log scale)")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment B: running time vs N  (n = 10,000)")
    ax.grid(alpha=0.3, which="both"); ax.legend()
    fig.tight_layout(); fig.savefig("chart_experiment_B.png", dpi=150)
    print("Charts saved: chart_experiment_A.png, chart_experiment_B.png")
    plt.show()
```

```
# program starts here: tests, then experiments, then graphs
if __name__ == "__main__":
    run_tests()
    res = benchmark()
    make_charts(res)
```

For each line in a graph we give a list of x values and a list of y values taken from the results, with times multiplied by 1000 to get milliseconds. The format strings "o-", "s-", "^-" and "x--" choose the marker and line style. In Graph 1 the y-axis is limited so that insertion sort does not squeeze the other lines. In Graph 2 semilogx puts the x-axis on a log scale because N goes from 10 to 1,000,000. Both graphs are saved as PNG files and shown with plt.show(). If matplotlib is not installed, a message is printed and the graphs are skipped. The last block starts the program.

Complete Source Code:

```python
# P-12.61: Bucket-sort for integer keys in the range [0, N-1], O(n + N) time
# Data Structures Lab Project

import random   # random test data and random lists for the experiments
import time     # time.perf_counter() is used as a stopwatch


# ---------------------------------------------------------------
# Data structure: one entry = (key, value)
# ---------------------------------------------------------------
class Entry:
    __slots__ = ("key", "value")   # only these two fields, saves memory

    def __init__(self, key, value=None):
        self.key = key
        self.value = value

    # how an entry looks when printed, e.g. (3, 'c1')
    def __repr__(self):
        return f"({self.key}, {self.value!r})"


# ---------------------------------------------------------------
# Main algorithm: bucket-sort
# entries : list to sort (sorted in place)
# N       : keys are in the range 0 .. N-1
# key     : how to get the key from an item (default: e.key)
# ---------------------------------------------------------------
def bucket_sort(entries, N, key=lambda e: e.key):
    if N < 2:
        raise ValueError("N must be at least 2")

    # create N empty buckets, B[k] holds all entries with key k   -> O(N)
    B = [[] for _ in range(N)]

    # scatter: put every entry at the end of its bucket           -> O(n)
    for e in entries:
        k = key(e)
        if not (0 <= k < N):
            raise ValueError(f"key {k} is outside the range [0, {N - 1}]")
        B[k].append(e)

    # gather: read buckets 0 .. N-1 in order back into the list   -> O(n + N)
    i = 0
    for bucket in B:
        for e in bucket:
            entries[i] = e
            i += 1

    return entries


# ---------------------------------------------------------------
# Comparison sorts, used only to compare running times
# ---------------------------------------------------------------

# insertion sort: shift bigger items right, put cur in the gap   O(n^2)
def insertion_sort(a):
    for j in range(1, len(a)):
        cur = a[j]
        i = j
        while i > 0 and a[i - 1] > cur:
            a[i] = a[i - 1]
            i -= 1
        a[i] = cur
    return a


# merge sort: split in halves, sort each half, merge them        O(n log n)
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    # merge two sorted lists, i and j point to the front items
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
```

```
    return out


# quick sort: smaller / equal / bigger than the pivot           O(n log n) average
def quick_sort(a):
    if len(a) <= 1:
        return a[:]
    pivot = a[len(a) // 2]
    smaller = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    bigger = [x for x in a if x > pivot]
    return quick_sort(smaller) + equal + quick_sort(bigger)


# ---------------------------------------------------------------
# Correctness tests (assert stops the program if an answer is wrong)
# ---------------------------------------------------------------
def run_tests():
    print("=" * 64)
    print("CORRECTNESS TESTS")
    print("=" * 64)

    # Test 1: small example, also checks stability (c1, c2, c3 keep order)
    data = [Entry(3, "c1"), Entry(0, "a"), Entry(3, "c2"), Entry(1, "b"),
            Entry(2, "x"), Entry(0, "a2"), Entry(3, "c3")]
    print("Before :", data)
    bucket_sort(data, 4)
    print("After  :", data)
    assert [e.key for e in data] == [0, 0, 1, 2, 3, 3, 3]
    assert [e.value for e in data if e.key == 3] == ["c1", "c2", "c3"]
    print("Test 1 passed - sorted and STABLE (c1, c2, c3 kept their order)\n")

    # Test 2: 200 random lists compared with Python's sorted()
    for trial in range(200):
        N = random.randint(2, 50)
        n = random.randint(0, 100)
        nums = [random.randrange(N) for _ in range(n)]
        assert bucket_sort(nums[:], N, key=lambda x: x) == sorted(nums)
    print("Test 2 passed - 200 random lists match Python's sorted()")

    # Test 3: edge cases
    assert bucket_sort([], 5, key=lambda x: x) == []
    assert bucket_sort([4], 5, key=lambda x: x) == [4]
    assert bucket_sort([1, 1, 1], 2, key=lambda x: x) == [1, 1, 1]
    assert bucket_sort([4, 3, 2, 1, 0], 5, key=lambda x: x) == [0, 1, 2, 3, 4]
    assert bucket_sort([0, 1, 2, 3], 4, key=lambda x: x) == [0, 1, 2, 3]
    print("Test 3 passed - empty, single, duplicates, reversed, sorted")

    # Test 4: wrong input must raise ValueError
    for bad_call in (lambda: bucket_sort([5], 5, key=lambda x: x),
                     lambda: bucket_sort([-1], 5, key=lambda x: x),
                     lambda: bucket_sort([0], 1, key=lambda x: x)):
        try:
            bad_call()
            raise AssertionError("should have raised ValueError")
        except ValueError:
            pass
    print("Test 4 passed - out-of-range keys and N < 2 raise ValueError\n")


# ---------------------------------------------------------------
# Performance analysis
# ---------------------------------------------------------------

# run func 3 times on a fresh copy of data and return the best time (seconds)
def time_it(func, data, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        copy = data[:]
        start = time.perf_counter()
        func(copy)
        best = min(best, time.perf_counter() - start)
    return best


def benchmark():
    random.seed(42)   # same random data on every run
    results = {}

    # Experiment A: N fixed at 1000, number of entries n grows
    print("=" * 64)
    print("EXPERIMENT A: vary n (N = 1000 fixed) - times in milliseconds")
    print("=" * 64)
```

```
    N = 1000
    sizes = [1000, 2000, 5000, 10000, 20000, 50000, 100000]
    print(f"{'n':>8} {'Bucket':>10} {'Merge':>10} {'Quick':>10} {'Insertion':>10} {'sorted()':>10}")
    rows_a = []
    for n in sizes:
        data = [random.randrange(N) for _ in range(n)]   # n random keys in 0..N-1
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        tq = time_it(quick_sort, data)
        # insertion sort is O(n^2), so it is only run for small n
        ti = time_it(insertion_sort, data, repeats=1) if n <= 5000 else None
        ts = time_it(sorted, data)
        rows_a.append((n, tb, tm, tq, ti, ts))
        ins = f"{ti*1000:10.2f}" if ti is not None else f"{'(skip)':>10}"
        print(f"{n:>8} {tb*1000:10.2f} {tm*1000:10.2f} {tq*1000:10.2f} {ins} {ts*1000:10.2f}")
    results["A"] = rows_a

    # Experiment B: n fixed at 10000, key range N grows
    print()
    print("=" * 64)
    print("EXPERIMENT B: vary N (n = 10000 fixed) - bucket sort, ms")
    print("=" * 64)
    n = 10000
    print(f"{'N':>10} {'Bucket (ms)':>12} {'Merge (ms)':>12}")
    rows_b = []
    for N in [10, 100, 1000, 10000, 100000, 1000000]:
        data = [random.randrange(N) for _ in range(n)]
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        rows_b.append((N, tb, tm))
        print(f"{N:>10} {tb*1000:12.2f} {tm*1000:12.2f}")
    results["B"] = rows_b
    print()
    return results


# draw the two graphs with matplotlib, save them as PNG and show them
def make_charts(results):
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed - charts skipped (pip install matplotlib)")
        return

    # Graph 1: Experiment A (time vs n)
    rows = results["A"]
    n = [r[0] for r in rows]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.plot(n, [r[1] * 1000 for r in rows], "o-", label="Bucket sort  O(n+N)", lw=2.5)
    ax.plot(n, [r[2] * 1000 for r in rows], "s-", label="Merge sort  O(n log n)")
    ax.plot(n, [r[3] * 1000 for r in rows], "^-", label="Quick sort  O(n log n)")
    # insertion sort only has times for small n, skip the None values
    ins = [(r[0], r[4] * 1000) for r in rows if r[4] is not None]
    ax.plot([x for x, _ in ins], [y for _, y in ins], "x--", label="Insertion sort  O(n²)")
    ax.set_xlabel("Number of entries n")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment A: running time vs n  (N = 1000)")
    # limit the y-axis so insertion sort does not squeeze the other lines
    ax.set_ylim(0, max(r[2] for r in rows) * 1000 * 1.15)
    ax.grid(alpha=0.3); ax.legend()
    fig.tight_layout(); fig.savefig("chart_experiment_A.png", dpi=150)

    # Graph 2: Experiment B (time vs N), x-axis on log scale
    rows = results["B"]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.semilogx([r[0] for r in rows], [r[1] * 1000 for r in rows], "o-", lw=2.5, label="Bucket sort")
    ax.semilogx([r[0] for r in rows], [r[2] * 1000 for r in rows], "s-", label="Merge sort")
    ax.set_xlabel("Key range N (log scale)")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment B: running time vs N  (n = 10,000)")
    ax.grid(alpha=0.3, which="both"); ax.legend()
    fig.tight_layout(); fig.savefig("chart_experiment_B.png", dpi=150)
    print("Charts saved: chart_experiment_A.png, chart_experiment_B.png")
    plt.show()


# program starts here: tests, then experiments, then graphs
if __name__ == "__main__":
    run_tests()
    res = benchmark()
    make_charts(res)
```

5. Results

5.1 Program Output (screenshot):

```
$ python bucket_sort.py
============================================================
CORRECTNESS TESTS
============================================================
Before : [(3, 'c1'), (0, 'a'), (3, 'c2'), (1, 'b'), (2, 'x'), (0, 'a2'), (3, 'c3')]
After  : [(0, 'a'), (0, 'a2'), (1, 'b'), (2, 'x'), (3, 'c1'), (3, 'c2'), (3, 'c3')]
Test 1 passed - sorted and STABLE (c1, c2, c3 kept their order)

Test 2 passed - 200 random lists match Python's sorted()
Test 3 passed - empty, single, duplicates, reversed, sorted
Test 4 passed - out-of-range keys and N < 2 raise ValueError

============================================================
EXPERIMENT A: vary n (N = 1000 fixed) - times in milliseconds
============================================================
       n     Bucket      Merge      Quick  Insertion   sorted()
    1000       0.31       2.32       1.61      27.85       0.08
    2000       0.41       3.77       2.91      91.45       0.18
    5000       0.73       9.88       5.93     529.28       0.63
   10000       2.07      18.56      11.03     (skip)       1.27
   20000       2.58      48.18      23.43     (skip)       2.74
   50000       6.36     114.42      56.40     (skip)       9.24
  100000      12.63     240.43     110.69     (skip)      14.85

============================================================
EXPERIMENT B: vary N (n = 10000 fixed) - bucket sort, ms
============================================================
         N  Bucket (ms)   Merge (ms)
        10         0.94        16.78
       100         1.02        17.21
      1000         1.41        19.39
     10000         2.42        18.65
    100000        13.65        23.16
   1000000       175.81        18.00

Charts saved: chart_experiment_A.png, chart_experiment_B.png
```

The program ran without any error. All four tests passed, both experiment tables were printed, and the two graphs were saved as chart_experiment_A.png and chart_experiment_B.png and opened on the screen.

5.2 Sample Input and Output:

	Entries (key, value)
Input	(3,'c1') (0,'a') (3,'c2') (1,'b') (2,'x') (0,'a2') (3,'c3')
Output	(0,'a') (0,'a2') (1,'b') (2,'x') (3,'c1') (3,'c2') (3,'c3')

The keys come out in order 0, 0, 1, 2, 3, 3, 3, and the three entries with key 3 (c1, c2, c3) keep the same order as in the input, so the sort is stable. A plain list works the same way, for example bucket_sort([4, 1, 3, 1, 0], 5, key=lambda x: x) returns [0, 1, 1, 3, 4].

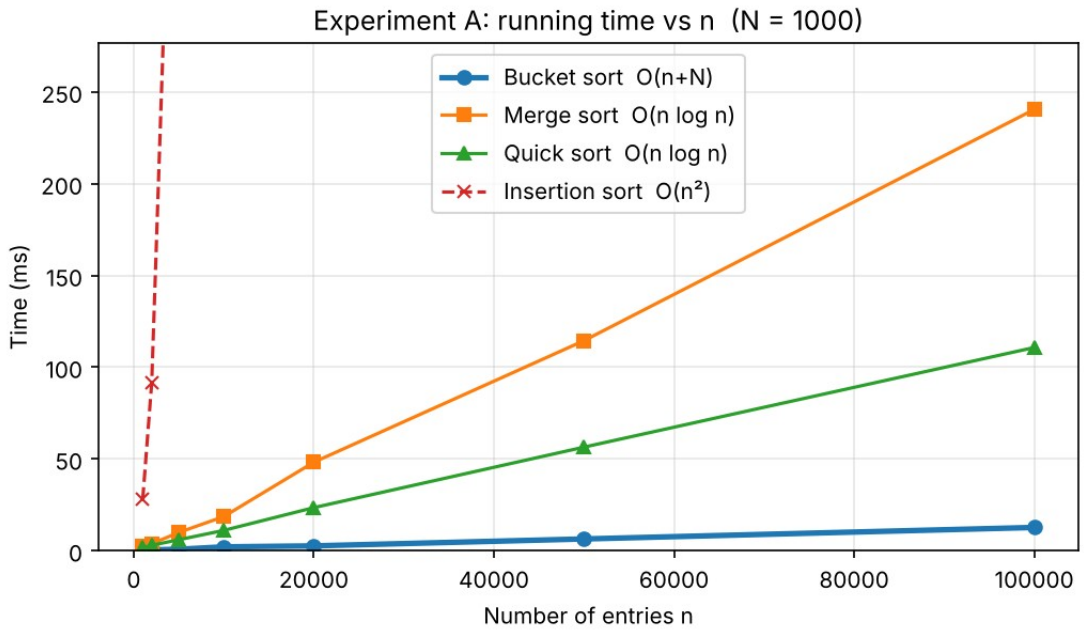
5.3 Test Results:

Test	What it checks	Result
1	Small example of 7 entries: keys sorted and equal keys keep their order (stable).	Passed
2	200 random lists with random n and N compared with Python's sorted().	Passed
3	Edge cases: empty list, one item, all equal, reversed, already sorted.	Passed
4	Wrong input: key equal to N, negative key, N = 1. Each must raise ValueError.	Passed

5.4 Experiment A: increasing n (N = 1000)

n	Bucket	Merge	Quick	Insertion	sorted()
1,000	0.31	2.32	1.61	27.85	0.08
2,000	0.41	3.77	2.91	91.45	0.18
5,000	0.73	9.88	5.93	529.28	0.63
10,000	2.07	18.56	11.03	skipped	1.27
20,000	2.58	48.18	23.43	skipped	2.74
50,000	6.36	114.42	56.40	skipped	9.24
100,000	12.63	240.43	110.69	skipped	14.85

Table 1: Running times in milliseconds

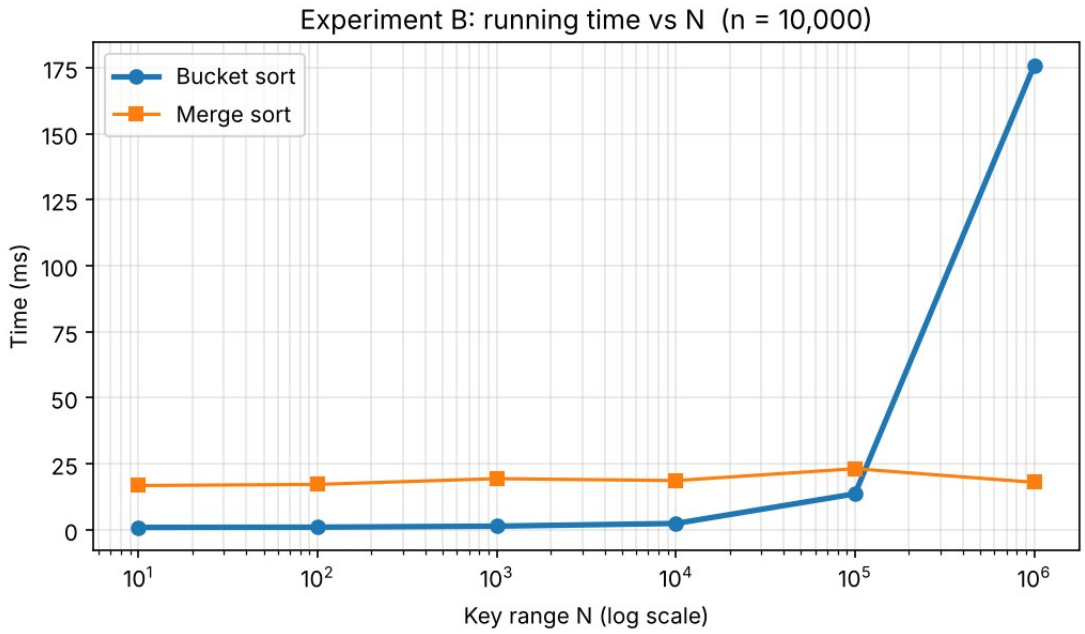


Graph 1: Running time vs n (N = 1000)

5.5 Experiment B: increasing N (n = 10,000)

N (key range)	Bucket-sort (ms)	Merge sort (ms)
10	0.94	16.78
100	1.02	17.21
1,000	1.41	19.39
10,000	2.42	18.65
100,000	13.65	23.16
1,000,000	175.81	18.00

Table 2: Running times in milliseconds



Graph 2: Running time vs key range N (n = 10,000), x-axis on log scale

6. Discussion

6.1 Analysis of the Implementation:

In Graph 1, bucket-sort is the lowest line and grows almost as a straight line. When n doubles from 50,000 to 100,000 its time also roughly doubles (6.36 ms to 12.63 ms), which shows linear growth. At n = 100,000 it is about 19 times faster than merge sort and about 9 times faster than quick sort. Insertion sort goes off the top of the graph; its time grows about 3 to 6 times when n grows 2 to 2.5 times, which is the $O(n^2)$ pattern. Quick sort was faster than merge sort here because there are only 1000 different keys, so many values are equal to the pivot and go into the equal list in one step. Python's sorted() is written in C, so it is very fast, but our pure Python bucket-sort still stays close to it.

In Graph 2, merge sort stays almost flat between 17 and 23 ms because it only compares keys and does not care about their range. Bucket-sort is very fast while N is smaller than or close to n. But when N becomes 1,000,000, it has to create and check one million buckets for only 10,000

entries, and most of them are empty. At that point it is about 10 times slower than merge sort. This shows that the "+N" part of O(n + N) is a real cost. So bucket-sort is the right choice when the keys are integers in a small range, like marks from 0 to 100, and not when the range is huge, like phone numbers.

6.2 Challenges Faced:

- At first it was confusing to understand how bucket-sort can sort without comparing anything. Doing a dry run on paper with a few buckets made it clear.
- Our first version of quick sort used a stack and swaps and was hard to follow, so we replaced it with a simpler version that splits the list into smaller, equal and bigger parts.
- The two graph files were first named chart_vary_n.png and chart_vary_N.png. On Windows these names are treated as the same file, so one graph overwrote the other. We renamed them to chart_experiment_A.png and chart_experiment_B.png.
- Small running times change a little on every run because of other programs on the computer. Taking the best of three runs made the results more stable.

6.3 Limitations:

- Bucket-sort only works for integer keys in a known range. It does not work directly for decimal numbers, strings or negative keys.
- If N is much larger than n, most buckets stay empty and the algorithm wastes time and memory.
- It needs O(n + N) extra memory for the buckets, so it is not an in-place algorithm.
- The exact running times depend on the computer. On another laptop the numbers will be different, but the pattern stays the same.

6.4 Lessons Learned:

- Big-O is not only theory; it can be checked by measuring real running times.
- There is no single best sorting algorithm. The right choice depends on the data.
- Testing with random data and edge cases gives confidence that the code is correct.
- Graphs make it much easier to see and explain the difference between algorithms.

7. Future Work

- Extend bucket-sort to decimal (float) keys by sorting the items inside each bucket.
- Support negative keys or keys that start from a value other than 0 by shifting them (subtracting the minimum key).
- Use our stable bucket-sort to build radix sort, which can sort large integers digit by digit.
- Improve quick sort so it works in place and uses less memory.
- Run each experiment more times and report the average with the best time.
- Implement the same algorithms in C++ or Java and compare their speed with Python.

8. Conclusion

In this project we implemented bucket-sort for integer keys in the range [0, N−1] and showed that it runs in O(n + N) time. Our tests showed that it is correct and stable. The experiments

showed that when the key range is small compared to the number of entries, bucket-sort is much faster than comparison based sorts like merge sort and quick sort. They also showed its weakness: when the key range becomes very large, the empty buckets make it slower than merge sort. So the best sorting algorithm depends on the data. Through this project we learned how theory like Big-O can be checked with real measurements, and how to choose an algorithm by looking at the type of keys we have.

9. References

1. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley. Section 12.4.2 (Bucket-Sort). ISBN 978-1-118-29027-9. https://www.oreilly.com/library/view/data-structures-and/9781118290279/

2. Python documentation, time.perf_counter(): https://docs.python.org/3/library/time.html#time.perf_counter

3. Python documentation, random module: https://docs.python.org/3/library/random.html

4. Matplotlib documentation, pyplot: https://matplotlib.org/stable/api/pyplot_summary.html

5. Matplotlib documentation, Axes.semilogx: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.semilogx.html

Appendix: Source Code (copy and run)

Requirements: Python 3 and matplotlib (install with: pip install matplotlib). Copy the code below into a file named bucket_sort.py and run it with: python bucket_sort.py

```
# P-12.61: Bucket-sort for integer keys in the range [0, N-1], O(n + N) time
# Data Structures Lab Project

import random   # random test data and random lists for the experiments
import time     # time.perf_counter() is used as a stopwatch


# ------------------------------------------------------------
# Data structure: one entry = (key, value)
# ------------------------------------------------------------
class Entry:
    __slots__ = ("key", "value")   # only these two fields, saves memory

    def __init__(self, key, value=None):
        self.key = key
        self.value = value

    # how an entry looks when printed, e.g. (3, 'c1')
    def __repr__(self):
        return f"({self.key}, {self.value!r})"


# ------------------------------------------------------------
# Main algorithm: bucket-sort
# entries : list to sort (sorted in place)
# N       : keys are in the range 0 .. N-1
# key     : how to get the key from an item (default: e.key)
# ------------------------------------------------------------
def bucket_sort(entries, N, key=lambda e: e.key):
    if N < 2:
        raise ValueError("N must be at least 2")

    # create N empty buckets, B[k] holds all entries with key k   -> O(N)
    B = [[] for _ in range(N)]

    # scatter: put every entry at the end of its bucket           -> O(n)
    for e in entries:
        k = key(e)
        if not (0 <= k < N):
            raise ValueError(f"key {k} is outside the range [0, {N - 1}]")
        B[k].append(e)

    # gather: read buckets 0 .. N-1 in order back into the list   -> O(n + N)
    i = 0
    for bucket in B:
        for e in bucket:
            entries[i] = e
            i += 1

    return entries


# ------------------------------------------------------------
```

```
# Comparison sorts, used only to compare running times
# ------------------------------------------------------------

# insertion sort: shift bigger items right, put cur in the gap   O(n^2)
def insertion_sort(a):
    for j in range(1, len(a)):
        cur = a[j]
        i = j
        while i > 0 and a[i - 1] > cur:
            a[i] = a[i - 1]
            i -= 1
        a[i] = cur
    return a


# merge sort: split in halves, sort each half, merge them        O(n log n)
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    # merge two sorted lists, i and j point to the front items
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out


# quick sort: smaller / equal / bigger than the pivot            O(n log n) average
def quick_sort(a):
    if len(a) <= 1:
        return a[:]
    pivot = a[len(a) // 2]
    smaller = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    bigger = [x for x in a if x > pivot]
    return quick_sort(smaller) + equal + quick_sort(bigger)


# ------------------------------------------------------------
# Correctness tests (assert stops the program if an answer is wrong)
# ------------------------------------------------------------
def run_tests():
    print("=" * 64)
    print("CORRECTNESS TESTS")
    print("=" * 64)

    # Test 1: small example, also checks stability (c1, c2, c3 keep order)
    data = [Entry(3, "c1"), Entry(0, "a"), Entry(3, "c2"), Entry(1, "b"),
            Entry(2, "x"), Entry(0, "a2"), Entry(3, "c3")]
    print("Before :", data)
    bucket_sort(data, 4)
    print("After  :", data)
    assert [e.key for e in data] == [0, 0, 1, 2, 3, 3, 3]
```

```
    assert [e.value for e in data if e.key == 3] == ["c1", "c2", "c3"]
    print("Test 1 passed - sorted and STABLE (c1, c2, c3 kept their order)\n")

    # Test 2: 200 random lists compared with Python's sorted()
    for trial in range(200):
        N = random.randint(2, 50)
        n = random.randint(0, 100)
        nums = [random.randrange(N) for _ in range(n)]
        assert bucket_sort(nums[:], N, key=lambda x: x) == sorted(nums)
    print("Test 2 passed - 200 random lists match Python's sorted()")

    # Test 3: edge cases
    assert bucket_sort([], 5, key=lambda x: x) == []
    assert bucket_sort([4], 5, key=lambda x: x) == [4]
    assert bucket_sort([1, 1, 1], 2, key=lambda x: x) == [1, 1, 1]
    assert bucket_sort([4, 3, 2, 1, 0], 5, key=lambda x: x) == [0, 1, 2, 3, 4]
    assert bucket_sort([0, 1, 2, 3], 4, key=lambda x: x) == [0, 1, 2, 3]
    print("Test 3 passed - empty, single, duplicates, reversed, sorted")

    # Test 4: wrong input must raise ValueError
    for bad_call in (lambda: bucket_sort([5], 5, key=lambda x: x),
                     lambda: bucket_sort([-1], 5, key=lambda x: x),
                     lambda: bucket_sort([0], 1, key=lambda x: x)):
        try:
            bad_call()
            raise AssertionError("should have raised ValueError")
        except ValueError:
            pass
    print("Test 4 passed - out-of-range keys and N < 2 raise ValueError\n")


# ------------------------------------------------------------------
# Performance analysis
# ------------------------------------------------------------------

# run func 3 times on a fresh copy of data and return the best time (seconds)
def time_it(func, data, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        copy = data[:]
        start = time.perf_counter()
        func(copy)
        best = min(best, time.perf_counter() - start)
    return best


def benchmark():
    random.seed(42)   # same random data on every run
    results = {}

    # Experiment A: N fixed at 1000, number of entries n grows
    print("=" * 64)
    print("EXPERIMENT A: vary n (N = 1000 fixed) - times in milliseconds")
    print("=" * 64)
    N = 1000
    sizes = [1000, 2000, 5000, 10000, 20000, 50000, 100000]
    print(f"{'n':>8} {'Bucket':>10} {'Merge':>10} {'Quick':>10} {'Insertion':>10}
{'sorted()':>10}")
```

```
    rows_a = []
    for n in sizes:
        data = [random.randrange(N) for _ in range(n)]   # n random keys in 0..N-1
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        tq = time_it(quick_sort, data)
        # insertion sort is O(n^2), so it is only run for small n
        ti = time_it(insertion_sort, data, repeats=1) if n <= 5000 else None
        ts = time_it(sorted, data)
        rows_a.append((n, tb, tm, tq, ti, ts))
        ins = f"{ti*1000:10.2f}" if ti is not None else f"{'(skip)':>10}"
        print(f"{n:>8} {tb*1000:10.2f} {tm*1000:10.2f} {tq*1000:10.2f} {ins}
{ts*1000:10.2f}")
    results["A"] = rows_a

    # Experiment B: n fixed at 10000, key range N grows
    print()
    print("=" * 64)
    print("EXPERIMENT B: vary N (n = 10000 fixed) - bucket sort, ms")
    print("=" * 64)
    n = 10000
    print(f"{'N':>10} {'Bucket (ms)':>12} {'Merge (ms)':>12}")
    rows_b = []
    for N in [10, 100, 1000, 10000, 100000, 1000000]:
        data = [random.randrange(N) for _ in range(n)]
        tb = time_it(lambda a: bucket_sort(a, N, key=lambda x: x), data)
        tm = time_it(merge_sort, data)
        rows_b.append((N, tb, tm))
        print(f"{N:>10} {tb*1000:12.2f} {tm*1000:12.2f}")
    results["B"] = rows_b
    print()
    return results


# draw the two graphs with matplotlib, save them as PNG and show them
def make_charts(results):
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed - charts skipped (pip install matplotlib)")
        return

    # Graph 1: Experiment A (time vs n)
    rows = results["A"]
    n = [r[0] for r in rows]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.plot(n, [r[1] * 1000 for r in rows], "o-", label="Bucket sort  O(n+N)", lw=2.5)
    ax.plot(n, [r[2] * 1000 for r in rows], "s-", label="Merge sort  O(n log n)")
    ax.plot(n, [r[3] * 1000 for r in rows], "^-", label="Quick sort  O(n log n)")
    # insertion sort only has times for small n, skip the None values
    ins = [(r[0], r[4] * 1000) for r in rows if r[4] is not None]
    ax.plot([x for x, _ in ins], [y for _, y in ins], "x--", label="Insertion sort  O(n²)")
    ax.set_xlabel("Number of entries n")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment A: running time vs n  (N = 1000)")
    # limit the y-axis so insertion sort does not squeeze the other lines
    ax.set_ylim(0, max(r[2] for r in rows) * 1000 * 1.15)
    ax.grid(alpha=0.3); ax.legend()
```

```
    fig.tight_layout(); fig.savefig("chart_experiment_A.png", dpi=150)

    # Graph 2: Experiment B (time vs N), x-axis on log scale
    rows = results["B"]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    ax.semilogx([r[0] for r in rows], [r[1] * 1000 for r in rows], "o-", lw=2.5,
label="Bucket sort")
    ax.semilogx([r[0] for r in rows], [r[2] * 1000 for r in rows], "s-", label="Merge sort")
    ax.set_xlabel("Key range N (log scale)")
    ax.set_ylabel("Time (ms)")
    ax.set_title("Experiment B: running time vs N  (n = 10,000)")
    ax.grid(alpha=0.3, which="both"); ax.legend()
    fig.tight_layout(); fig.savefig("chart_experiment_B.png", dpi=150)
    print("Charts saved: chart_experiment_A.png, chart_experiment_B.png")
    plt.show()


# program starts here: tests, then experiments, then graphs
if __name__ == "__main__":
    run_tests()
    res = benchmark()
    make_charts(res)
```